

ML Loop Code-Along - Step 10

Scaling: stop big-number features taking over

Incremental prerequisite prep for Imperial ML/AI coursework

Prepared for Anthony Watts - June 2026

Rule: nothing appears before it is explained.

10. Scaling: stop big-number features taking over

Some models care about distance. K-nearest neighbours is one of them. If one feature is salary in pounds and another is age in years, the salary column can dominate unless features are scaled.

Step 10.1: Import a distance-based classifier

Why this cell exists

KNeighborsClassifier predicts by looking at nearby training examples. Because nearby means distance, feature scale matters.

```
from sklearn.neighbors import KNeighborsClassifier
```

What to look for

This model still uses fit() and predict(), but fit mainly stores the training examples.

Step 10.2: Try KNN without scaling

Why this cell exists

This gives us a baseline. It is not necessarily awful on every dataset, but it is vulnerable when feature scales differ.

```
knn_plain = KNeighborsClassifier(n_neighbors=5)
knn_plain.fit(X_cls_train, y_cls_train)

knn_plain_predictions = knn_plain.predict(X_cls_test)

print("Unscaled KNN accuracy:", round(
    accuracy_score(y_cls_test, knn_plain_predictions), 3
))
```

What to look for

The key idea is not this exact score. The key idea is that KNN uses distances between rows.

Step 10.3: Import the scaler

Why this cell exists

StandardScaler standardises each feature by removing the training-set mean and scaling by the training-set standard deviation.

```
from sklearn.preprocessing import StandardScaler
```

What to look for

This object learns scaling numbers from data. That means it must be fitted carefully.

Step 10.4: Create the scaler object

Why this cell exists

Like a model, the scaler starts blank. It has not learned feature means or standard deviations yet.

```
scaler = StandardScaler()
```

What to look for

The next step is where the scaler learns from training data.

Step 10.5: Fit the scaler on training data only

Why this cell exists

This is important. The scaler must not learn from the test set, because that would leak information from evaluation data into training workflow.

```
scaler.fit(X_cls_train)
```

What to look for

The scaler now knows training-set means and standard deviations. It has not used y, because scaling only concerns X.

Step 10.6: Transform train and test features

Why this cell exists

Use the training-fitted scaler to transform both training and test features. Do not fit a separate scaler on the test set.

```
X_cls_train_scaled = scaler.transform(X_cls_train)
X_cls_test_scaled = scaler.transform(X_cls_test)
```

What to look for

The output is now NumPy arrays rather than pandas DataFrames. That is normal for many scikit-learn transformations.

Step 10.7: Train KNN on scaled features

Why this cell exists

Now distance calculations are less likely to be dominated by whichever feature had the largest raw units.

```
knn_scaled = KNeighborsClassifier(n_neighbors=5)
knn_scaled.fit(X_cls_train_scaled, y_cls_train)

knn_scaled_predictions = knn_scaled.predict(X_cls_test_scaled)

print("Scaled KNN accuracy:", round(
    accuracy_score(y_cls_test, knn_scaled_predictions), 3
))
```

What to look for

Scaling is not a magic guarantee. It is a necessary hygiene step for many distance-based or gradient-based models.